

Assignment = 4

Section - A

1) Transaction in DBMS

A transaction is a logical unit of work that consists of multiple database operations (like, insert, update, delete) executed as a single unit. It ensures that either all operations are completed successfully or none are applied, maintaining database consistency.

2) ACID properties

- Atomicity : All or nothing execution
- Consistency : Database moves from one valid state to another.
- Isolation : Transaction don't interfere with each other.
- Durability : Once committed, changes are permanent.

3) Concurrency Control

It is the mechanism used to manage multiple transaction executing simultaneously without causing inconsistency. It ensures correctness while improving system performance.

4) Deadlock

A deadlock occurs when two or more transaction wait indefinitely for each others resources and none can proceed.

5) Serializability

It ensures that concurrent execution of transaction gives the same result as if they were executed

Serially (one after another).

6) Recovery in DBMS

Recovery is the process of restoring the database to a consistent state after a failure using logs, undo and redo operations.

7) Check Point

A check point is a mechanism that saves the current state of the database, reducing recovering time by avoiding representing the entire log.

Section - B

1) ACID with Banking Example

In a money transfer ($A \rightarrow B$):-

- * Atomicity $\rightarrow$ Debit and credit must both happen.
- * Consistency $\rightarrow$ Total balance remains unchanged.
- * Isolation $\rightarrow$ No other transaction sees partial update.
- * Durability $\rightarrow$ Once completed, data is permanent stored.

2) Dead Lock Situation

Eg $\rightarrow$

- * Transaction T_1 locks Account A, waits for B.
- * Transaction T_2 locks Account B, waits for A.

Both wait forever $\rightarrow$ DeadLock

3) Serial vs concurrent Execution

- * Serial Execution $\rightarrow$ Transactions run one by one.

Concurrent Execution $\rightarrow$ Multiple transactions run simultaneously.

$\rightarrow$ Concurrent is faster but needs control.

$\rightarrow$ Serial is safe but slower.

Section - C

1) Banking Transfer & Acid.

* Atomicity : Either full transfer or none.

* Consistency $\rightarrow$ No loss of money.

* Isolation $\rightarrow$ other users don't see intermediate state.

* Durability $\rightarrow$ final balance persists.

2) Two transaction updating same data.

$\rightarrow$ Possible issues:-

* Lost update

* Dirty Read

* Non - Repeatable Read.

These are concurrency problems solved using locking or isolation levels.

3) System crash During Transaction

i) check log file

ii) Undo incomplete transaction

iii) Redo committed transaction

iv) Restore consistent database state.